

UFRN - Universidade Federal do Rio Grande do Norte

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 13 - ARQUITETURAS PARALELAS

1. Enunciado

Avalie a escalabilidade do seu código de Navier-Stokes utilizando algum nó de computação do NPAD. Procure identificar gargalos de escalabilidade e reporte o seu progresso em versões sucessivas da evolução do código otimizado. Comente sobre a escalabilidade, a escalabilidade fraca e a escalabilidade forte das versões.

2. Metodologia

Como o enunciado requer um pouco do entendimento sobre a equação de Navier-Stokes para que seja possível entender a execução do código em C.

2.1. O que é a equação de Navier-Stokes

As equações de Navier-Stokes formam o conjunto fundamental de equações diferenciais parciais não lineares que descrevem o movimento de fluidos (líquidos e gases). Elas aplicam as leis de Newton da mecânica (especificamente a conservação do momento linear e da massa) a meios contínuos.

Componentes Principais da Equação de Momento: para um fluido incompressível com densidade constante, a formulação geral para o vetor velocidade $\mathbf{u}$ é expressa como:

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}$$

Figura 1 — Equação do fluido incompressível

Cada termo representa uma força física atuando sobre o fluido:

- $\partial \mathbf{u} / \partial t$: Aceleração local (como a velocidade muda com o tempo em um ponto fixo).

- $\rho(\mathbf{u} \cdot \nabla \mathbf{u})$: Advecção (ou convecção), que representa o transporte de momento pelo próprio fluxo do fluido (termo não linear responsável pela complexidade e turbulência).
- $-\nabla p$: Gradiente de pressão, as forças geradas por diferenças de pressão no fluido.
- $\mu \nabla^2 \mathbf{u}$: Termo viscoso (difusão de momento), correspondente ao atrito interno do fluido — o único efeito isolado e simulado no código de diferenças finitas deste trabalho.
- $\mathbf{f}$: Forças externas volumétricas, como a gravidade ou forças eletromagnéticas.

Equação da Continuidade: além do momento, as simulações de fluidos exigem a conservação da massa. Para fluidos incompressíveis, ela se resume a $\nabla \cdot \mathbf{u} = 0$ — a divergência do campo de velocidades deve ser nula em qualquer ponto (o que entra em um volume deve sair imediatamente).

Aplicações e Desafios Matemáticos: as aplicações cobrem a previsão do tempo, aerodinâmica de aeronaves, projeto de turbinas, hidrodinâmica naval e computação gráfica para simulação de fumaça e água. Do ponto de vista teórico, as equações de Navier-Stokes constituem um dos sete Problemas do Prêmio do Milênio do Clay Mathematics Institute: até hoje não se provou formalmente se, dadas condições iniciais suaves em 3D, sempre existirão soluções suaves para todo o tempo futuro, ou se o sistema pode desenvolver singularidades.

2.2 Execução

Os códigos serão executados levando em consideração o que já havia sido utilizado no script.sh para a tarefa anterior, aplicando as mudanças apenas ao adicionar o export com os parâmetros de afinidade de threads:

Script.sh

```
#!/bin/bash
#SBATCH --job-name=tarefa_12
#SBATCH --time=0-0:30
#SBATCH --partition=amd-512
#SBATCH --nodes=1
#SBATCH --exclusive
#SBATCH --cpus-per-task=32
#SBATCH --output=%x_%j.out

cd ~/Tarefa_12
source ~/pascal-releases-master/env.sh
export OMP_PROC_BIND=close
export OMP_PLACES=cores
gcc -fopenmp navier_stokes_p.c -lmpascalops -o navier_stokes_p_v0 -lm
```

```
pascalanalyzer ./navier_stokes_p_v0 -t man -g -r 5 --idtm 1 -c 1,2,4,8,16,32 -i  
10,20,40,80,160,320 -v INFO -o navier_stokes_v0.json
```

3 - Código

Inicialmente irei utilizar o código de v3 da tarefa anterior que utiliza collapse(3) no passo_difusão e utilizando o pascal_star(1) e paralelizando a função gaussiana e também aplicando o pascal_start(2).

```
#define _POSIX_C_SOURCE 199309L  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <math.h>  
#include <time.h>  
#include <omp.h>  
#include "pascalops.h"  
  
#define LX 1.0  
#define LY 1.0  
#define LZ 1.0  
#define NU 0.01  
  
#define AMPLITUDE 2.0  
#define LARGURA 0.03  
  
#define NX_PADRAO 64  
#define NY_PADRAO 64  
#define NZ_PADRAO 64  
#define N_STEPS_PADRAO 200  
#define CSV_PADRAO "difusao3d_p_collapse3.csv"  
#define MAX_PTS_PADRAO 48  
#define N_SNAPS_PADRAO 5
```

```

static double agora_em_segundos(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double) ts.tv_sec + (double) ts.tv_nsec / 1e9;
}

static inline long idx(int i, int j, int k, int NY, int NZ) {
    return ((long) i * NY + j) * NZ + k;
}

static double *aloca_campo(int NX, int NY, int NZ) {
    double *campo = (double *) calloc((size_t) NX * NY * NZ, sizeof(double));
    if (campo == NULL) {
        fprintf(stderr, "Erro: falha ao alocar memoria.\n");
        exit(EXIT_FAILURE);
    }
    return campo;
}

static void aplica_contorno(double *c, int NX, int NY, int NZ) {
    for (int i = 0; i < NX; i++)
        for (int j = 0; j < NY; j++) {
            c[idx(i, j, 0, NY, NZ)] = 0.0;
            c[idx(i, j, NZ - 1, NY, NZ)] = 0.0;
        }
    for (int i = 0; i < NX; i++)
        for (int k = 0; k < NZ; k++) {
            c[idx(i, 0, k, NY, NZ)] = 0.0;
            c[idx(i, NY - 1, k, NY, NZ)] = 0.0;
        }
}

```

```

for (int j = 0; j < NY; j++)
    for (int k = 0; k < NZ; k++) {
        c[idx(0, j, k, NY, NZ)] = 0.0;
        c[idx(NX - 1, j, k, NY, NZ)] = 0.0;
    }
}

static void passo_difusao(const double *c, double *cn, double dt, double dx, double dy,
double dz, int NX, int NY, int NZ) {
    const double kx = NU * dt / (dx * dx);
    const double ky = NU * dt / (dy * dy);
    const double kz = NU * dt / (dz * dz);

    /* Gerando a região Paralela */
    pascal_start(1);
    #pragma omp parallel
    {
        #pragma omp for collapse(2)
        for (int i = 1; i < NX - 1; i++) {
            for (int j = 1; j < NY - 1; j++) {
                for (int k = 1; k < NZ - 1; k++) {
                    long p = idx(i, j, k, NY, NZ);
                    double centro = c[p];
                    cn[p] = centro
                        + kx * (c[idx(i + 1, j, k, NY, NZ)] - 2.0 * centro + c[idx(i - 1, j, k, NY, NZ)])
                        + ky * (c[idx(i, j + 1, k, NY, NZ)] - 2.0 * centro + c[idx(i, j - 1, k, NY, NZ)])
                        + kz * (c[idx(i, j, k + 1, NY, NZ)] - 2.0 * centro + c[idx(i, j, k - 1, NY, NZ)]);
                }
            }
        }
    }
}

```

```

    }

}

pascal_stop(1);

}

/* Gaussian 3D no centro do dominio. */
static void aplica_perturbacao_gaussiana(double *u, double dx, double dy, double dz, int
NX, int NY, int NZ) {
    pascal_start(2);
    #pragma omp parallel for collapse(3)
    for (int i = 0; i < NX; i++)
        for (int j = 0; j < NY; j++)
            for (int k = 0; k < NZ; k++) {
                double x = i * dx - 0.5 * LX;
                double y = j * dy - 0.5 * LY;
                double z = k * dz - 0.5 * LZ;
                double r2 = x * x + y * y + z * z;
                u[idx(i, j, k, NY, NZ)] += AMPLITUDE * exp(-r2 / (2.0 * LARGURA *
LARGURA));
            }
    pascal_stop(2);
    aplica_contorno(u, NX, NY, NZ);
}

/* Grava um snapshot (subamostrado com passo ex,ey,ez) no CSV. */
static long grava_snapshot(FILE *f, const double *u, int NX, int NY, int NZ,
double dx, double dy, double dz, long passo, double t,

```

```

        int ex, int ey, int ez) {
long linhas = 0;
for (int i = 0; i < NX; i += ex)
    for (int j = 0; j < NY; j += ey)
        for (int k = 0; k < NZ; k += ez) {
            fprintf(f, "%ld,%.6g,%d,%d,%d,%.5f,%.5f,%.5f,%.6g\n",
                    passo, t, i, j, k, i * dx, j * dy, k * dz,
                    u[idx(i, j, k, NY, NZ)]);
            linhas++;
        }
return linhas;
}

int main(int argc, char **argv) {
    int NX = (argc > 1) ? atoi(argv[1]) : NX_PADRAO;
    int NY = (argc > 2) ? atoi(argv[2]) : NY_PADRAO;
    int NZ = (argc > 3) ? atoi(argv[3]) : NZ_PADRAO;
    long N_STEPS = (argc > 4) ? atol(argv[4]) : N_STEPS_PADRAO;
    const char *arq_csv = (argc > 5) ? argv[5] : CSV_PADRAO;
    int max_pts = (argc > 6) ? atoi(argv[6]) : MAX_PTS_PADRAO;
    int n_snaps = (argc > 7) ? atoi(argv[7]) : N_SNAPS_PADRAO;

    if (NX < 3 || NY < 3 || NZ < 3 || N_STEPS < 1 || max_pts < 2 || n_snaps < 2) {
        fprintf(stderr, "Uso: %s [NX>=3] [NY>=3] [NZ>=3] [N_STEPS>=1] [saida.csv]
[MX_PTS>=2] [N_SNAPS>=2]\n", argv[0]);
        return EXIT_FAILURE;
    }

    double dx = LX / (NX - 1);
    double dy = LY / (NY - 1);

```

```

double dz = LZ / (NZ - 1);

double limite_estabilidade = 0.5 / (NU * (1.0 / (dx * dx) + 1.0 / (dy * dy) + 1.0 / (dz * dz)));

double dt = 0.4 * limite_estabilidade;


double *u = aloca_campo(NX, NY, NZ);

double *u_novo = aloca_campo(NX, NY, NZ);

long total_pontos = (long) NX * NY * NZ;


aplica_perturbacao_gaussiana(u, dx, dy, dz, NX, NY, NZ);


FILE *f = fopen(arq_csv, "w");
if (f == NULL) {
    fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", arq_csv);
    return EXIT_FAILURE;
}

fprintf(f, "passo,t,i,j,k,x,y,z,u\n");


int ex = (NX + max_pts - 1) / max_pts;
int ey = (NY + max_pts - 1) / max_pts;
int ez = (NZ + max_pts - 1) / max_pts;


fprintf(stderr, "Malha = %d x %d x %d (%.2f milhoes de pontos) | passos = %ld |
Paralela | dt = %.3e\n", NX, NY, NZ, total_pontos / 1e6, N_STEPS, dt);


double tempo_calculo = 0.0; /* so os passos; a gravacao do CSV fica de fora */
long linhas = 0;
int s = 0;


for (long passo = 0; passo <= N_STEPS; passo++) {
    /* snapshots igualmente espacados entre o passo 0 e o final */

```

```

    if (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) {
        linhas += grava_snapshot(f, u, NX, NY, NZ, dx, dy, dz, passo, passo * dt, ex, ey,
ez);

        while (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) s++;
    }
    if (passo == N_STEPS) break;

    double inicio = agora_em_segundos();
    passo_difusao(u, u_novo, dt, dx, dy, dz, NX, NY, NZ);
    double *tmp = u; u = u_novo; u_novo = tmp; /* troca de ponteiros, sem memcpy */
    tempo_calculo += agora_em_segundos() - inicio;
}
fclose(f);

double soma = 0.0;
for (long p = 0; p < total_pontos; p++) soma += u[p];

fprintf(stderr, "(soma de u) = %.10f\n", soma);
fprintf(stderr, "[csv] %s: %d snapshots, %ld linhas (amostragem %dx%dx%d)\n",
arq_csv, s, linhas, ex, ey, ez);
printf("Tempo: %.6f s\n", tempo_calculo);

free(u);
free(u_novo);
return 0;
}

```

4 - Resultado

O primeiro teste realizado foi utilizando `export OMP_PROC_BIND=close` junto com `export OMP_PLACES=cores` fazendo com que as threads criadas, sejam criadas nas threads físicas e que estejam próximas das master usando o `script.sh`.

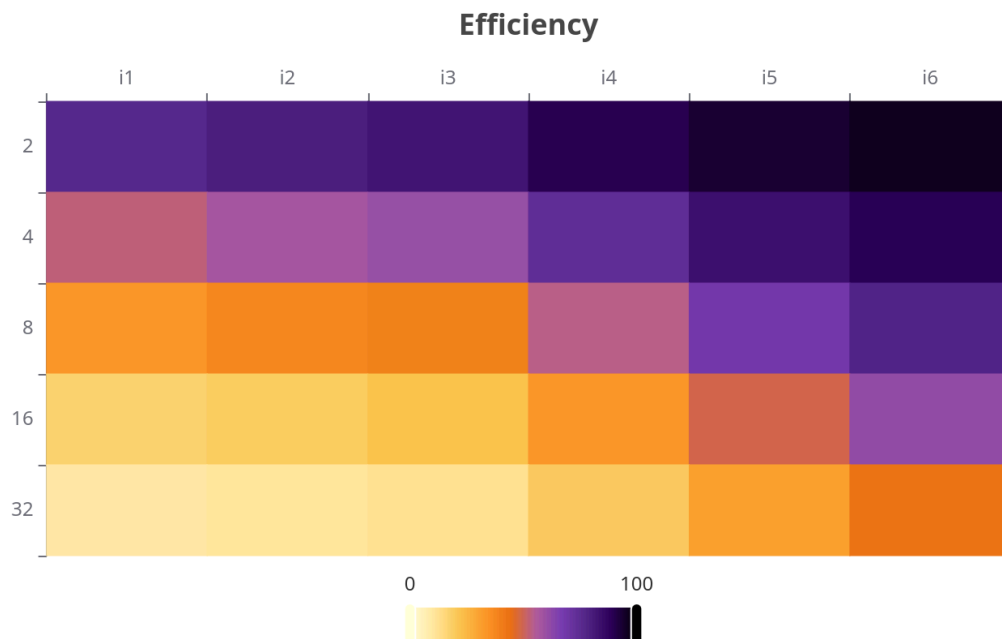


Figura 2 — Eficiência (Efficiency) — Teste 1 — `OMP_PROC_BIND=close`, `OMP_PLACES=cores`

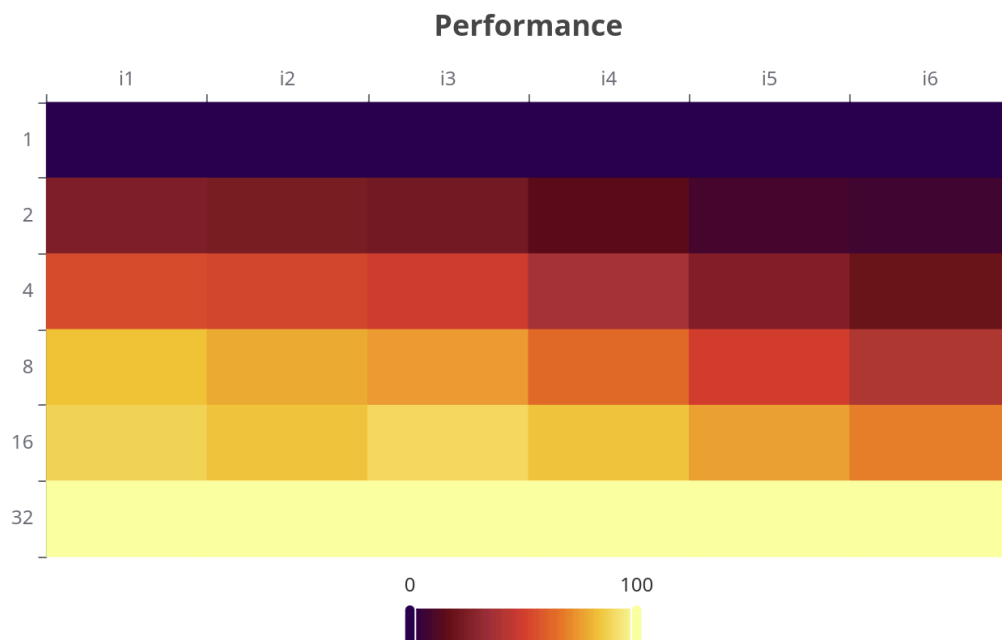


Figura 3 — Desempenho (Performance) — Teste 1 — `OMP_PROC_BIND=close`, `OMP_PLACES=cores`

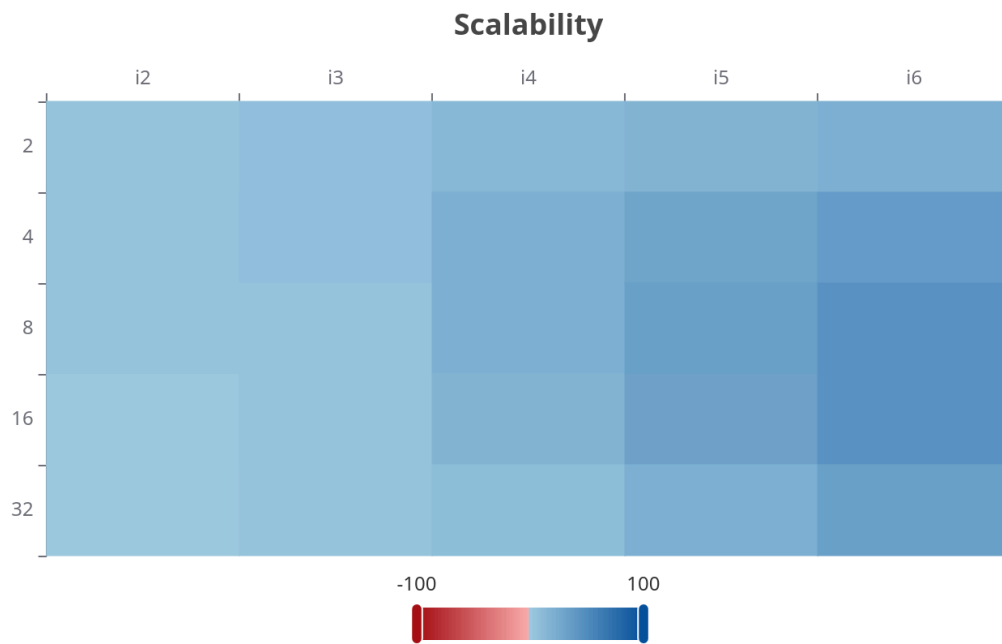


Figura 4 — Escalabilidade (Scalability) — Teste 1 — OMP_PROC_BIND=close, OMP_PLACES=cores

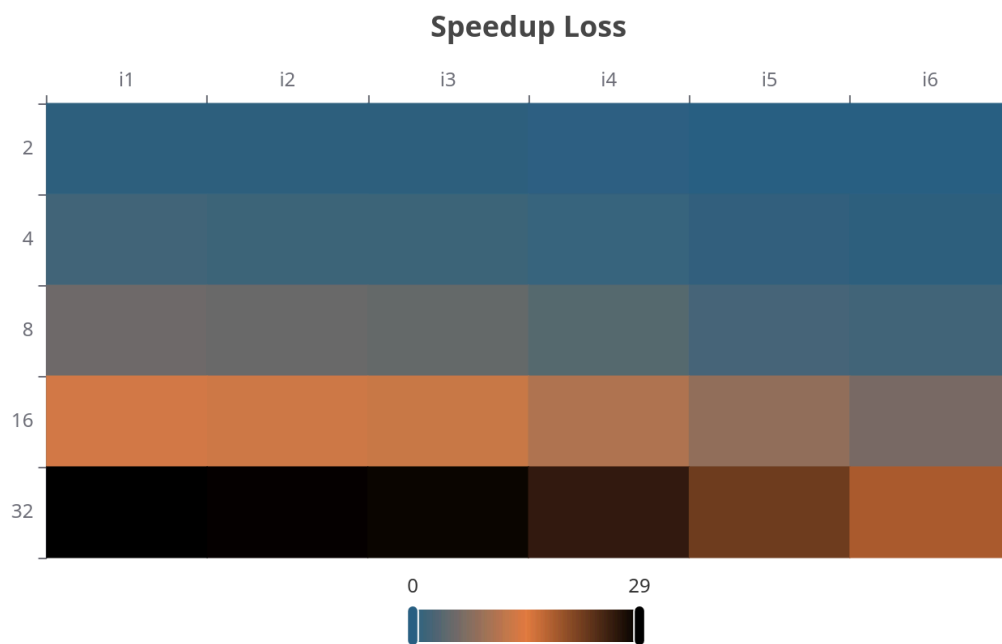


Figura 5 — Perda de Speedup (Speedup Loss) — Teste 1 — OMP_PROC_BIND=close, OMP_PLACES=cores

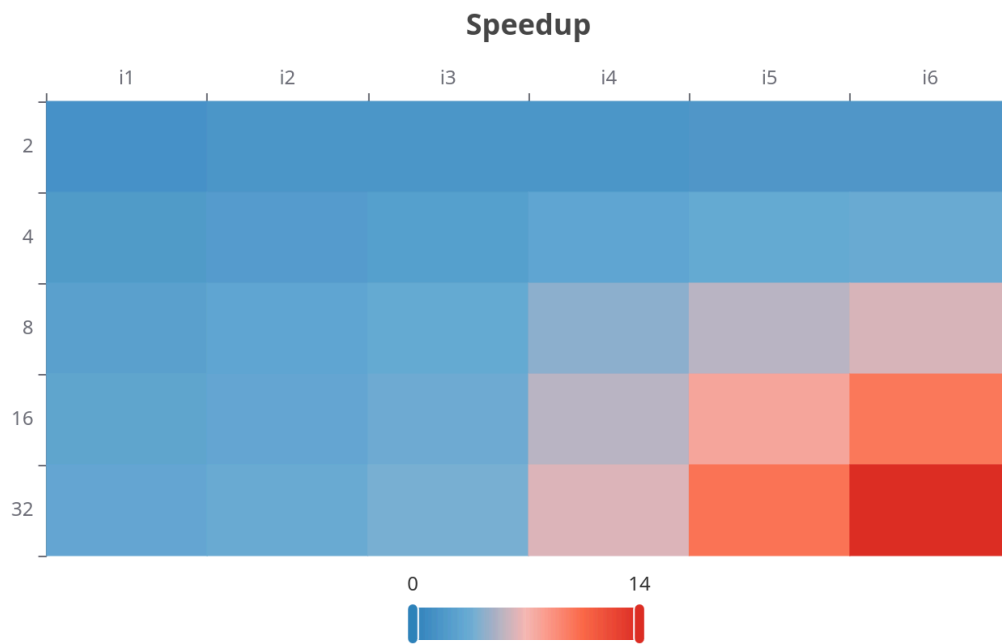


Figura 6 — Speedup — Teste 1 — OMP_PROC_BIND=close, OMP_PLACES=cores

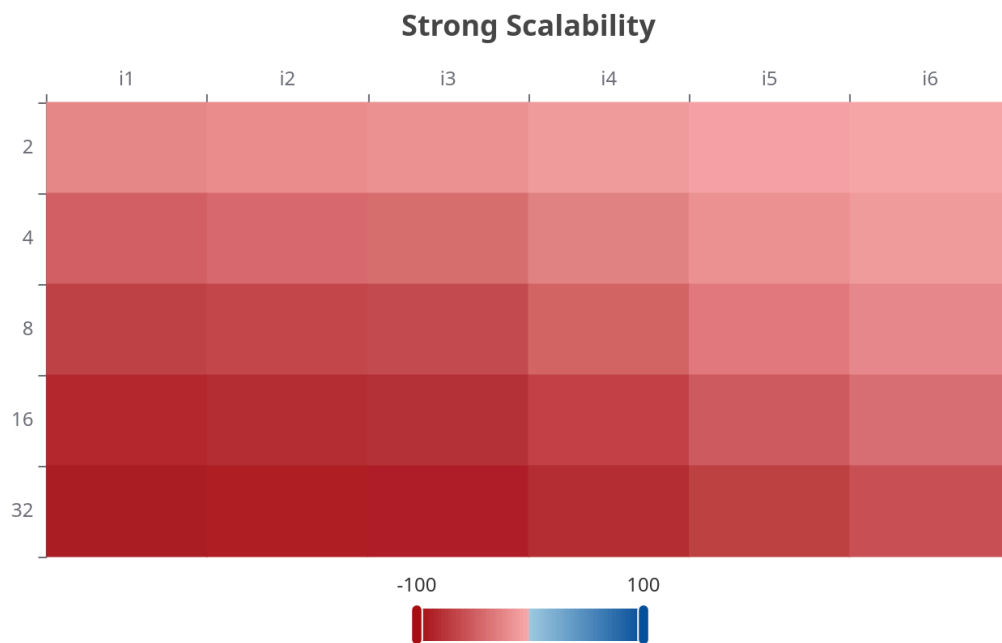


Figura 7 — Escalabilidade Forte (Strong Scalability) — Teste 1 — OMP_PROC_BIND=close, OMP_PLACES=cores

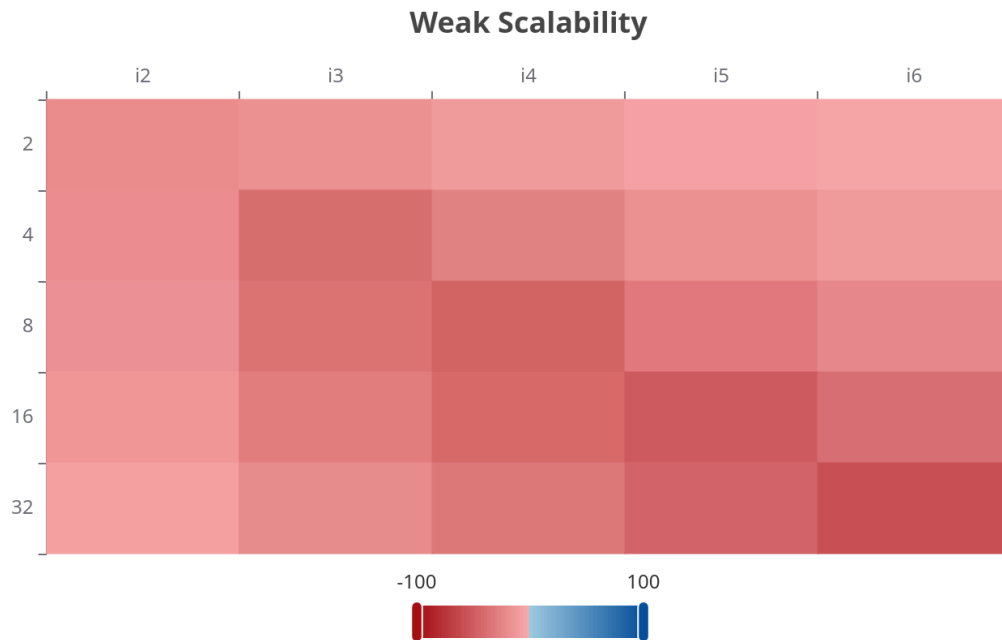


Figura 8 — Escalabilidade Fraca (Weak Scalability) — Teste 1 — `OMP_PROC_BIND=close`, `OMP_PLACES=cores`

Para o segundo teste foi feita a seguinte alteração no script.sh mantendo o mesmo código de antes `export OMP_PROC_BIND=spread`, `export OMP_PLACES=cores`, que nesse caso é espaça as tarefas nas threads físicas de maneira espaçada.

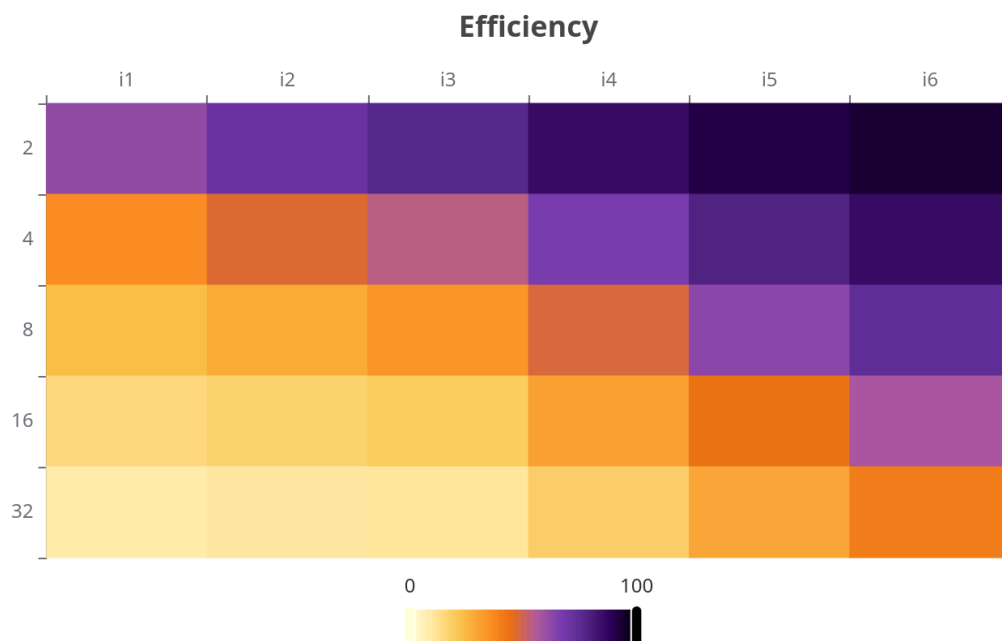


Figura 9 — Eficiência (Efficiency) — Teste 2 — `OMP_PROC_BIND=spread`, `OMP_PLACES=cores`

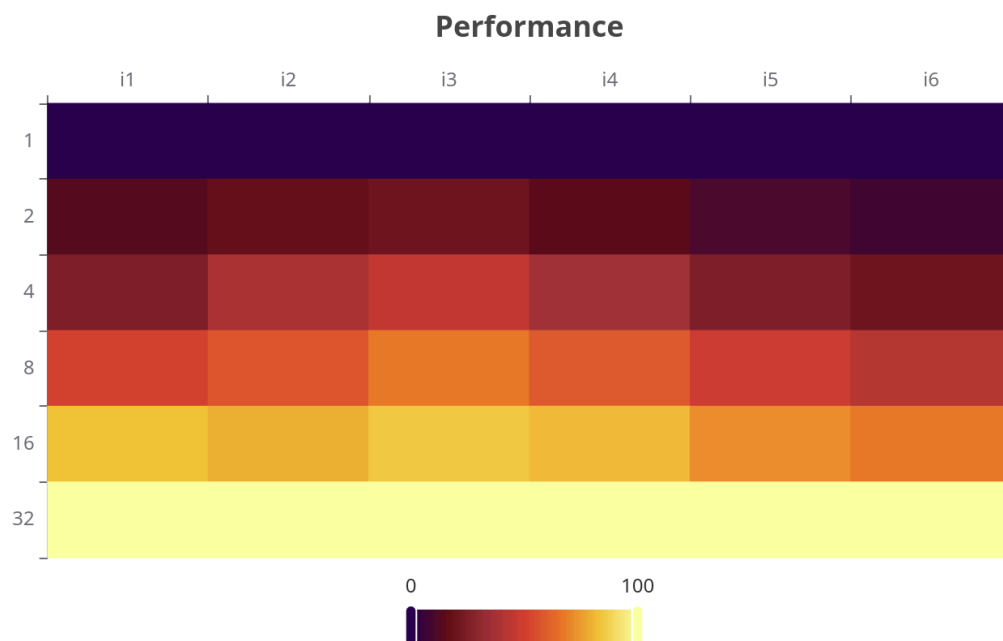


Figura 10 — Desempenho (Performance) — Teste 2 — OMP_PROC_BIND=spread,
OMP_PLACES=cores

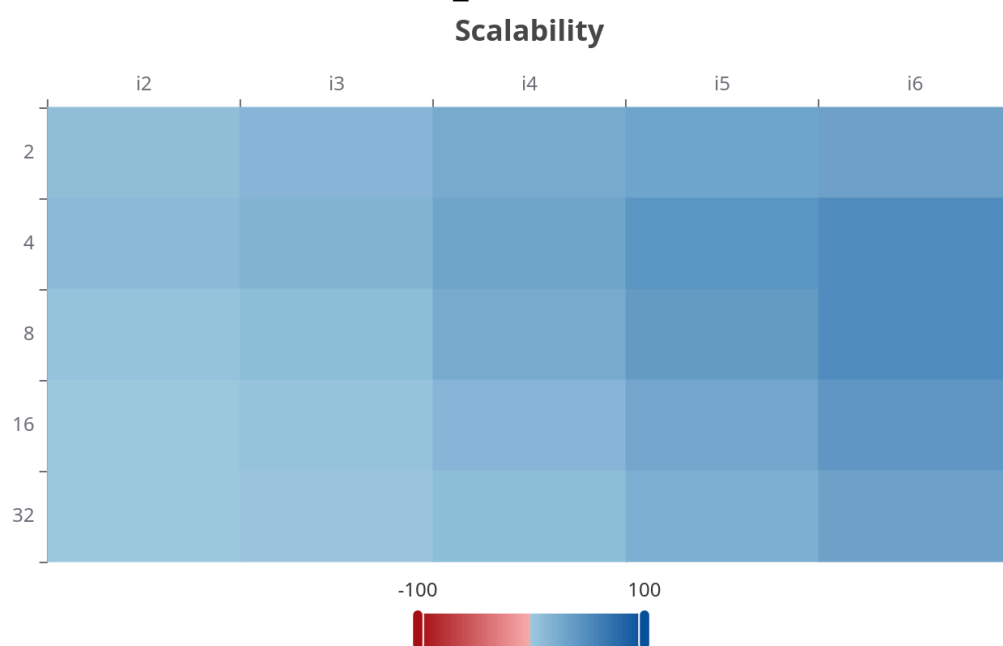


Figura 11 — Escalabilidade (Scalability) — Teste 2 — OMP_PROC_BIND=spread,
OMP_PLACES=cores

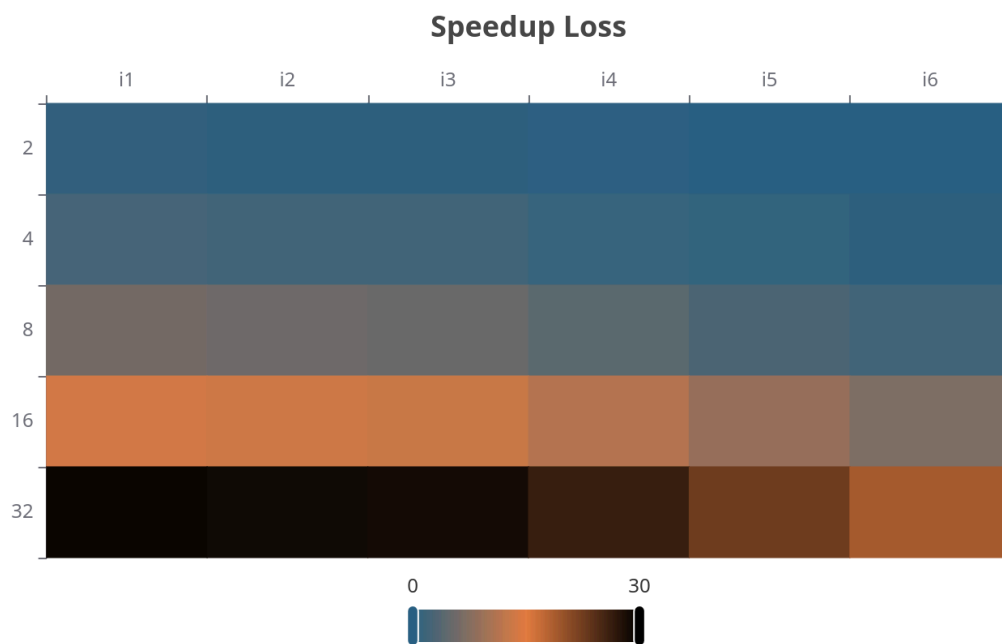


Figura 12 — Perda de Speedup (Speedup Loss) — Teste 2 — OMP_PROC_BIND=spread, OMP_PLACES=cores

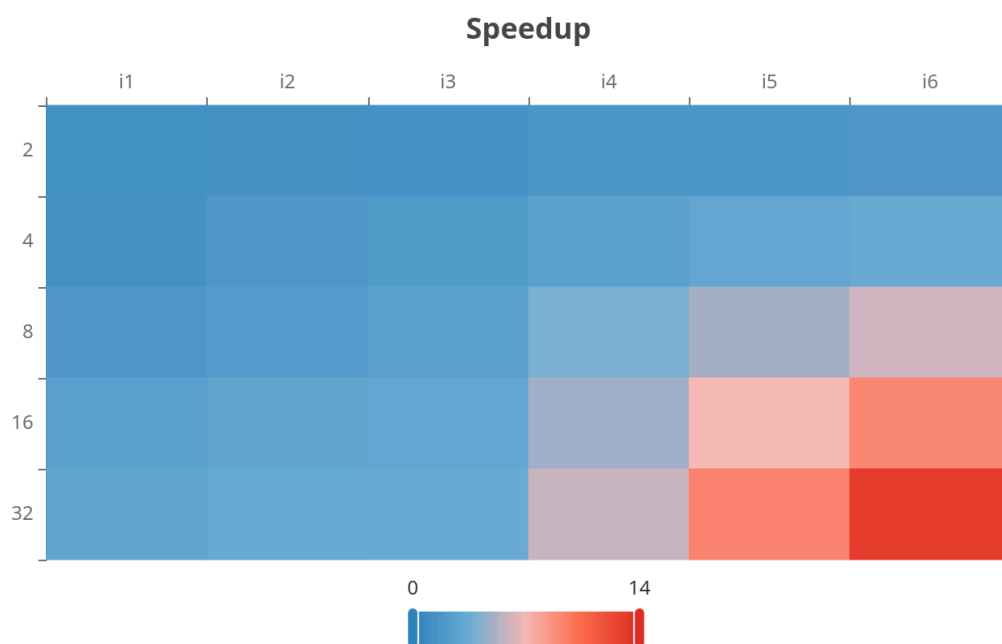


Figura 13 — Speedup — Teste 2 — OMP_PROC_BIND=spread, OMP_PLACES=cores

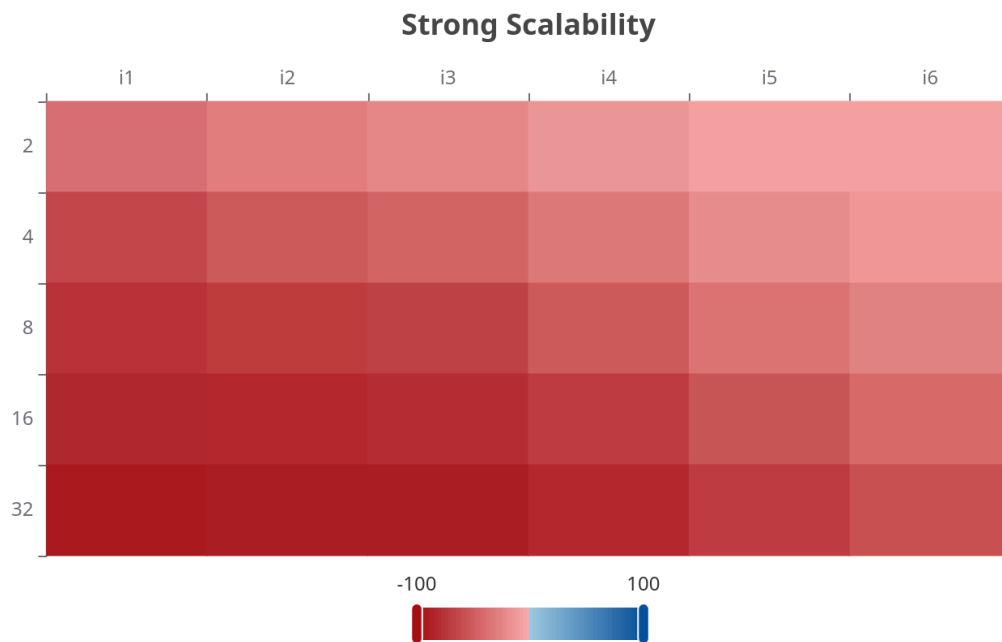


Figura 14 — Escalabilidade Forte (Strong Scalability) — Teste 2 — `OMP_PROC_BIND=spread`,
`OMP_PLACES=cores`

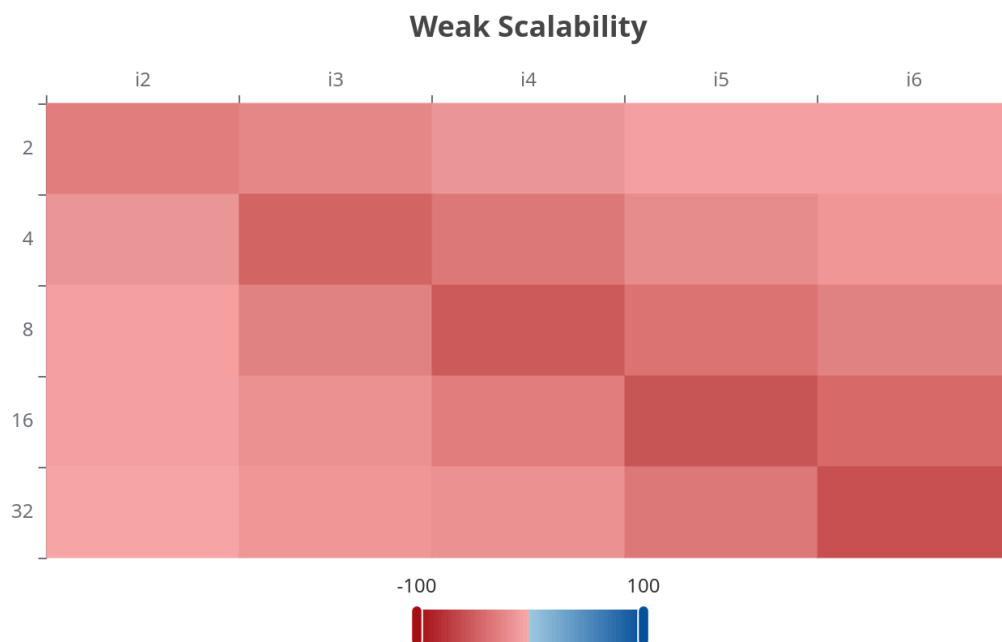
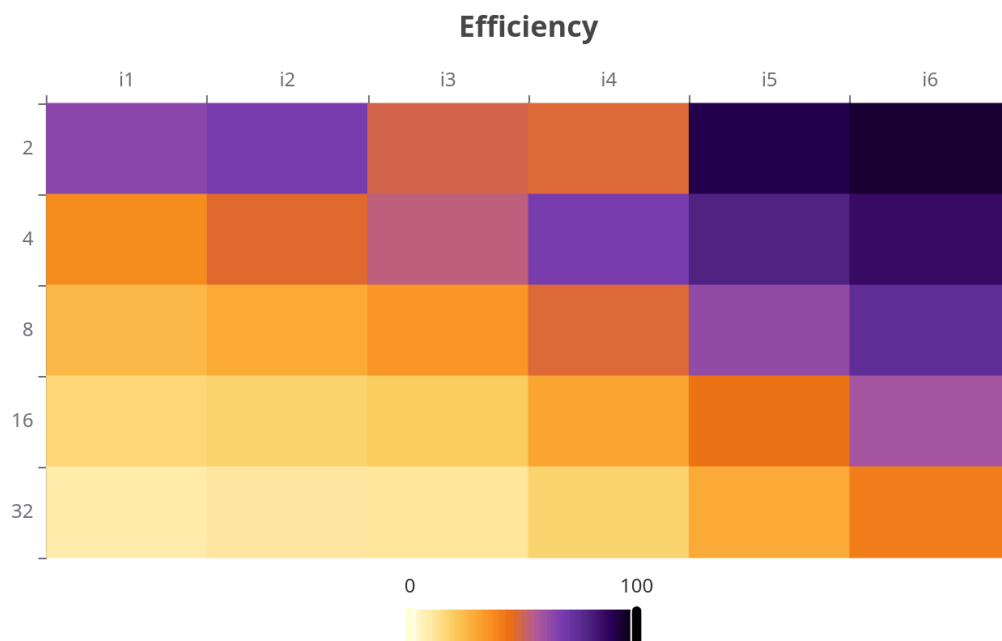
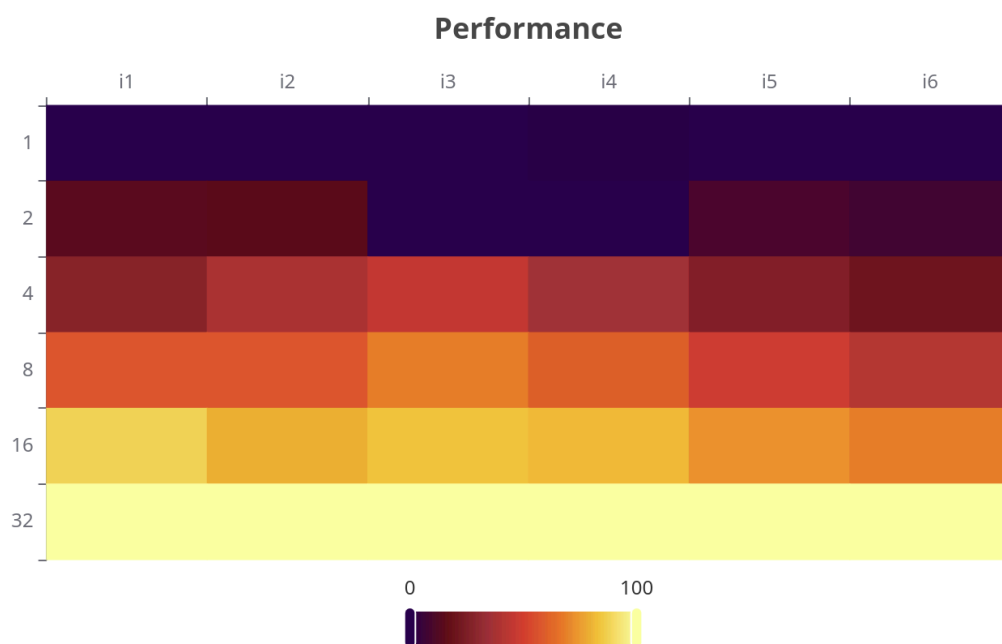


Figura 15 — Escalabilidade Fraca (Weak Scalability) — Teste 2 — `OMP_PROC_BIND=spread`,
`OMP_PLACES=cores`

Por ultimo um teste usando `spread` e `export OMP_PLACES="{0:16:1},{16:16:1}"`, que se divide em 2 grupos de 16 cores parecido com os exemplos dos slides.



*Figura 16 — Eficiência (Efficiency) — Teste 3 — OMP_PROC_BIND=spread,
OMP_PLACES="{0:16:1},{16:16:1}"*



*Figura 17 — Desempenho (Performance) — Teste 3 — OMP_PROC_BIND=spread,
OMP_PLACES="{0:16:1},{16:16:1}"*

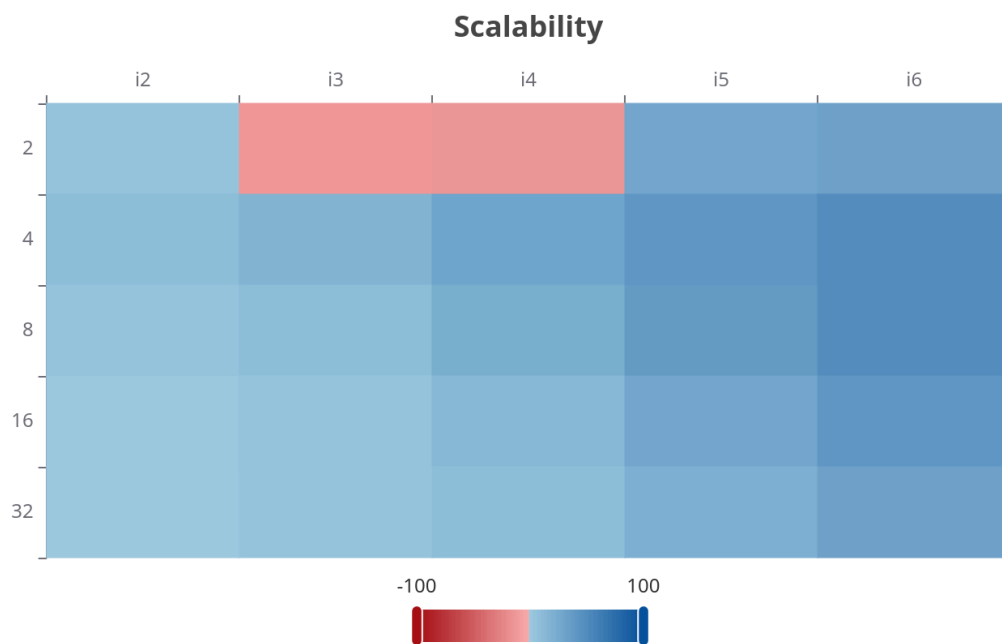


Figura 18 — Escalabilidade (Scalability) — Teste 3 — OMP_PROC_BIND=spread, OMP_PLACES="{0:16:1},{16:16:1}"

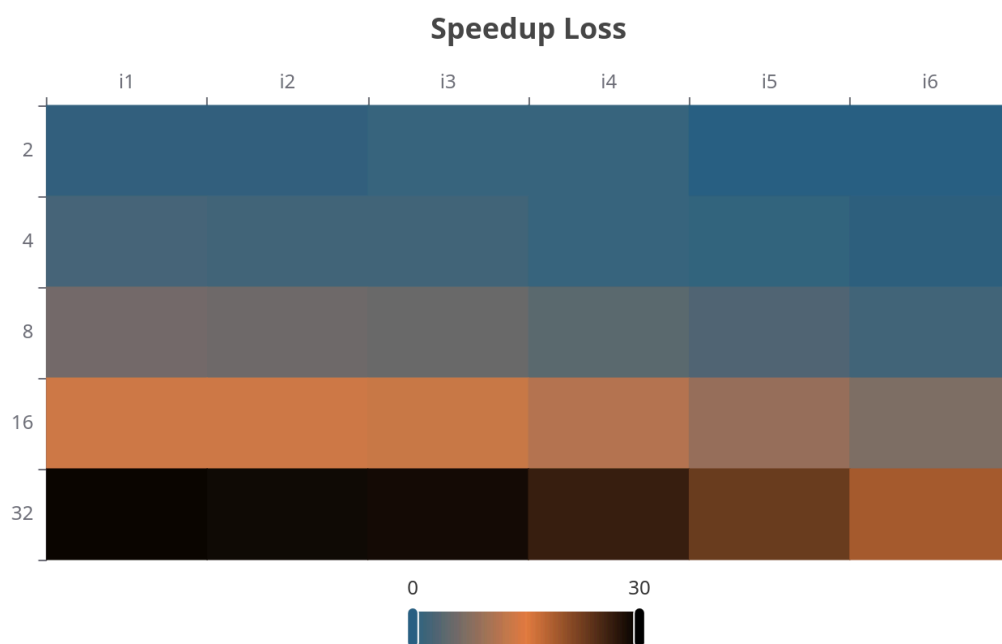


Figura 19 — Perda de Speedup (Speedup Loss) — Teste 3 — OMP_PROC_BIND=spread, OMP_PLACES="{0:16:1},{16:16:1}"

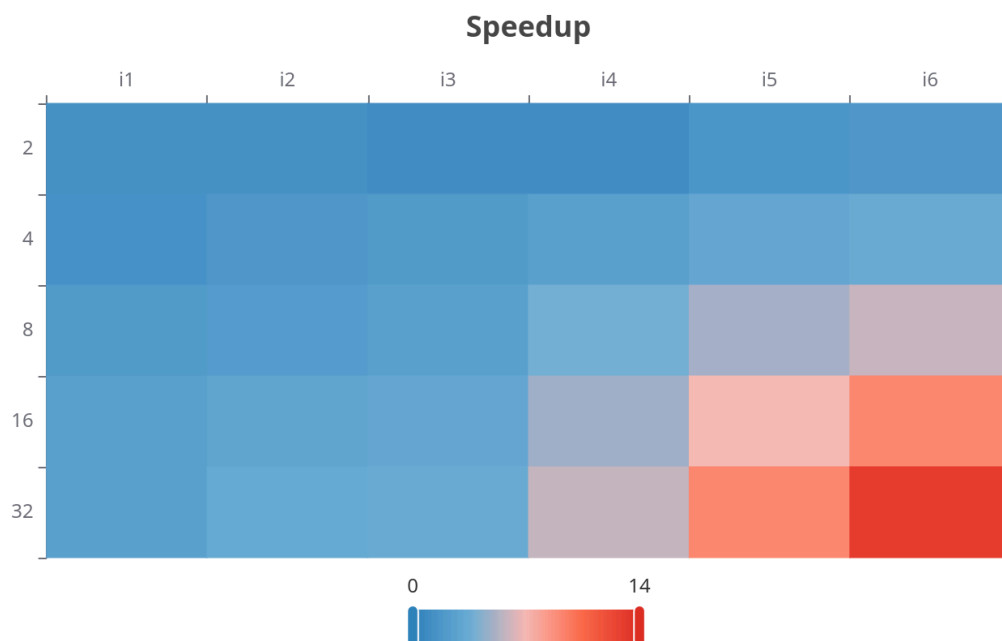


Figura 20 — Speedup — Teste 3 — OMP_PROC_BIND=spread, OMP_PLACES="{0:16:1},{16:16:1}"

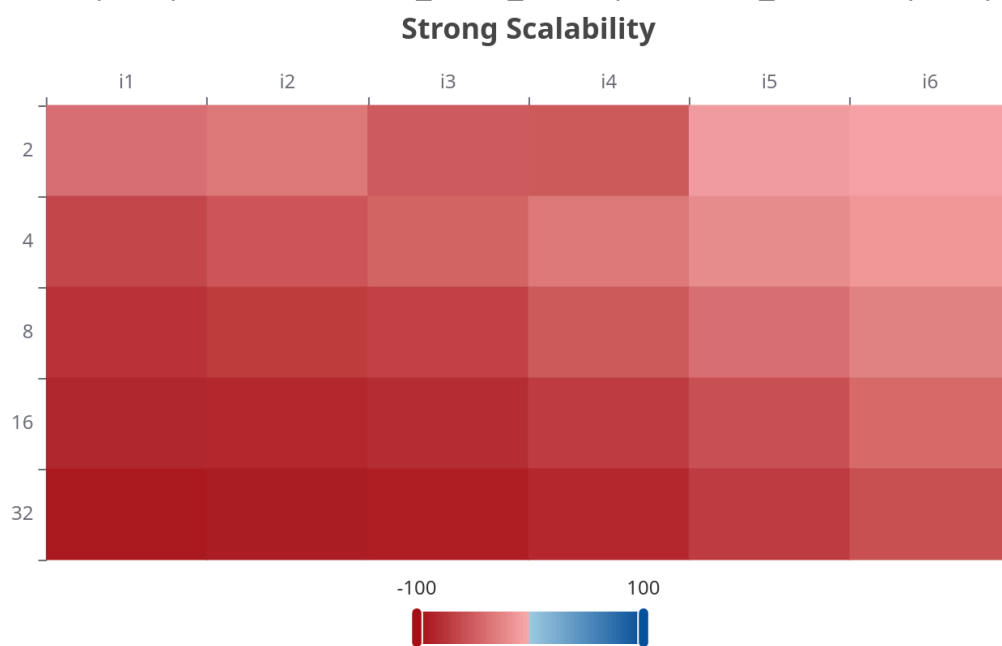


Figura 21 — Escalabilidade Forte (Strong Scalability) — Teste 3 — OMP_PROC_BIND=spread, OMP_PLACES="{0:16:1},{16:16:1}"

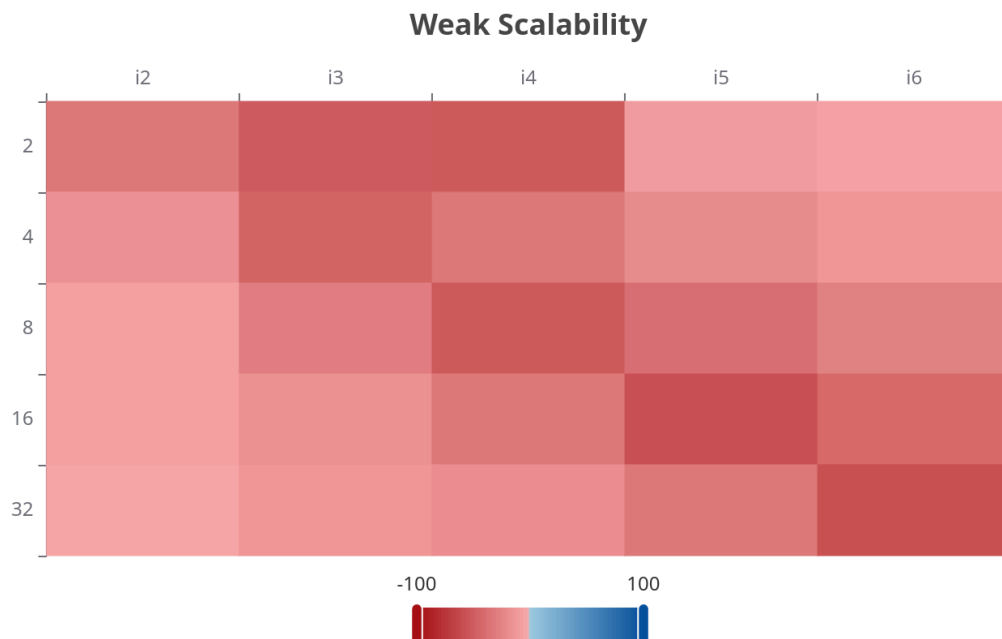


Figura 22 — Escalabilidade Fraca (Weak Scalability) — Teste 3 — `OMP_PROC_BIND=spread`,
`OMP_PLACES="{0:16:1},{16:16:1}"`

5 - Conclusão

Comparando os três testes de afinidade de threads (close/cores, spread/cores e spread com os grupos personalizados "{0:16:1},{16:16:1}"), os gráficos de Eficiência, Desempenho, Speedup, Perda de Speedup e Escalabilidade Forte apresentaram padrões praticamente idênticos entre si. Em todos os três casos o speedup máximo observado, atingido em 32 threads com a maior entrada (i6), ficou em torno de 14x sobre um ideal de 32x, e a Perda de Speedup atingiu seu pico (próximo de 29–30) exatamente na mesma combinação de 32 threads com as menores entradas (i1–i3).

Efeito da afinidade sobre a escalabilidade

A conclusão principal é que, para este código, a política de afinidade de threads não alterou de forma relevante a escalabilidade observada. Os mapas de Escalabilidade Forte (Strong Scalability) e Escalabilidade Fraca (Weak Scalability) mostram a mesma tendência nos três testes: eficiência decrescente conforme o número de threads aumenta, mais acentuada nas entradas menores, evidenciando que o gargalo do código não está na forma como os threads são distribuídos entre os núcleos físicos, e sim em outro fator limitante.

Isso é coerente com a natureza do kernel de difusão viscosa (esquema FTCS de 7 pontos): trata-se de um cálculo com baixa intensidade aritmética por byte acessado, ou seja, memory-bound. Como o nó do NPAD utilizado é exclusivo (--exclusive) e o SLURM já aloca os 32 cores inteiros para o job, tanto o agrupamento (close) quanto a distribuição (spread) dos threads acabam saturando a mesma largura de banda de memória agregada do nó conforme o número de threads cresce — por isso close, spread/cores e spread com grupos manuais de 16 núcleos convergem para curvas de speedup e eficiência praticamente sobrepostas.

A única diferença notável

A exceção observada foi no mapa de Escalabilidade do Teste 3 (spread com `OMP_PLACES="{0:16:1},{16:16:1}"`): para 2 threads nas entradas i3 e i4, o valor apareceu negativo (vermelho), enquanto nos Testes 1 e 2 essa mesma célula é sempre positiva (azul). Uma explicação plausível é que, ao fixar manualmente dois grupos de 16 núcleos, o runtime com spread coloca, já em 2 threads, uma thread em cada grupo — que provavelmente correspondem a domínios de memória diferentes (sockets/CCDs distintos) — introduzindo tráfego entre domínios que `OMP_PLACES=cores`, ao deixar o runtime decidir a topologia automaticamente, evita nesse mesmo ponto da curva. Esse efeito é pequeno e pontual, restrito a poucas células da tabela, mas indica que forçar manualmente a topologia sem antes confirmar os domínios físicos reais do nó (via `numactl -H`, por exemplo) pode introduzir uma pequena penalidade em contagens baixas de threads.

Piorou ou melhorou em relação ao esperado?

Nenhuma das três configurações de afinidade melhorou nem piorou de forma significativa a escalabilidade do código em relação às demais — os resultados foram, na prática, equivalentes. O gargalo de escalabilidade identificado (queda de eficiência a partir de 8–16 threads, mais acentuada quanto menor a entrada) é estrutural do kernel de difusão e do limite de banda de memória do nó, não um efeito da afinidade de threads escolhida. O único ponto de atenção prático é evitar agrupamentos manuais de `OMP_PLACES` que não reflitam a topologia NUMA real do nó, já que isso pode gerar pequenas penalidades isoladas sem trazer nenhum ganho de escalabilidade em compensação.